UNIVERSIDAD AUTÓNOMA DE ZACATECAS

Unidad Académica de Ingeniería Eléctrica | Licenciatura en Ingeniería en Computación / Software

Materia: Programación Orientada a Objetos 1 y Laboratorio | **Profesor:** Carlos Héctor Castañeda Ramírez

Actividad: Segundo Examen Parcial (60%)	**Proyecto:** p115_SegundoExamenParcial (Flota de Vehículos)
Estudiante: Alan	**Repositorio GitHub:** https://github.com/Alangeforce/POO-Programas
Modalidad: Asistido por IA (5 Fases Graduales)	**Fecha de Evaluación:** Octubre 2026

Matriz de Cumplimiento de la Metodología Gradual en 5 Fases

Fase	Enfoque Metodológico	Acción / Evidencia Asistida por IA	Criterio Rúbrica
Fase 1	Análisis y Modelado UML	Generación de diagrama UML (Mermaid/PlantUML) sin solicitar código Java. Verificación de herencia y composición.	10 pts (Diagrama)
Fase 2	Scaffolding (Esqueleto Base)	Estructura base de clases, constructores y getters/setters. Prohibición estricta de lógica de bonos y toString.	10 pts (Gradualidad)
Fase 3	Lógica y Polimorfismo	Método abstracto getBonoConductor(), sobreescritura @Override en subclases y cálculo de CostoTotal con super().	15 pts (Polimorfismo)
Fase 4	Integración y Gestor Flota	Implementación de Flota con List<Vehiculo>, acumulación polimórfica sin instanceof, for-each en reporte() y App main.	20 pts (Flota/Salida)
Fase 5	Code Review y Late Binding	Revisión de calidad con IA y redacción técnica en App.java sobre enlace dinámico (invokevirtual y vtable en JVM).	5 pts (Explicación)

Fase 1: Análisis y Modelado (Diagrama de Clases UML)

Objetivo de la Fase: Analizar el problema 'Flota de Vehículos' y generar el diagrama de clases en formato UML (Mermaid/PlantUML) respetando clases abstractas, herencia y agregación/composición, sin solicitar código fuente en Java.

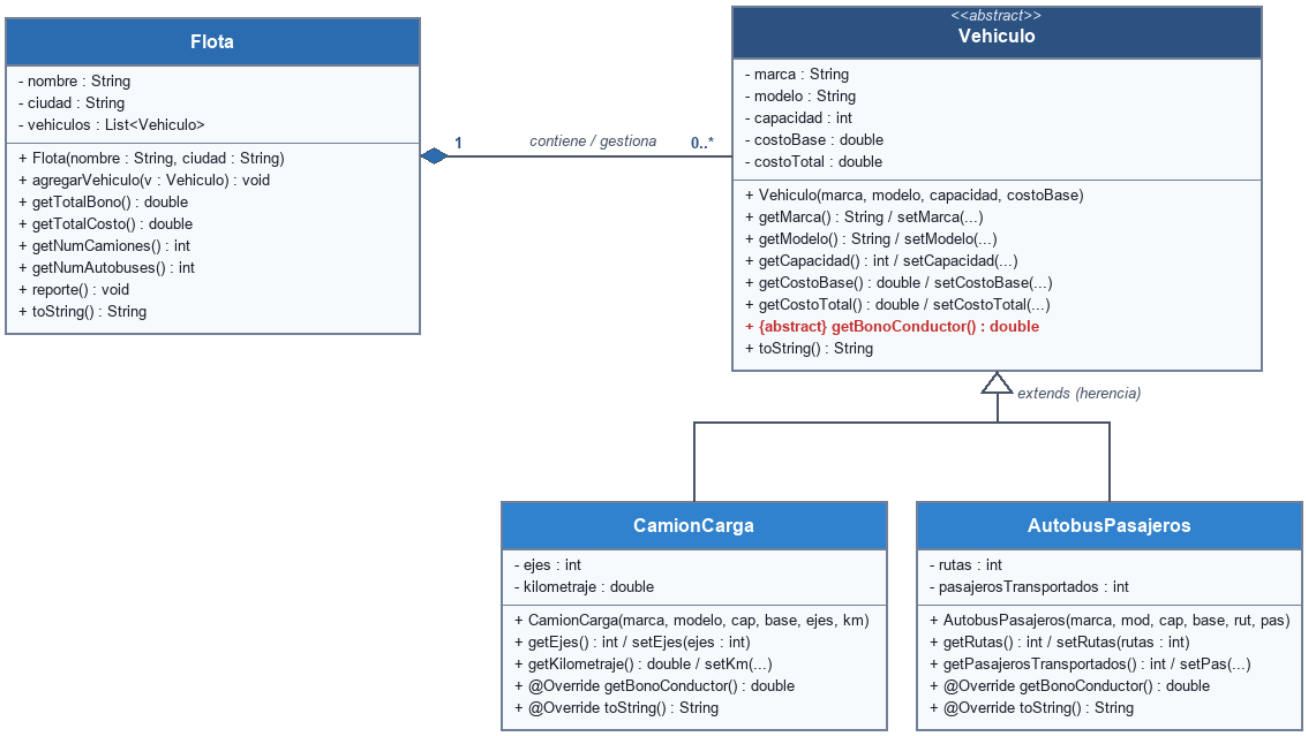
PROMPT UTILIZADO (Interacción con IA):

```
Actúa como un arquitecto de software senior especializado en UML y POO. Con base en la siguiente descripción del problema 'Flota de Vehículos', genera ÚNICAMENTE el diagrama de clases en formato Mermaid / PlantUML. RESTRICCIÓN CRÍTICA: NO generes código Java todavía. Requerimientos del diagrama: 1. Clase abstracta base 'Vehiculo' con atributos: Marca, Modelo, Capacidad, CostoBase, CostoTotal; y el método abstracto 'getBonoConductor() : double'. 2. Clases derivadas 'CamionCarga' (Ejes, Kilometraje) y 'AutobusPasajeros' (Rutas, PasajerosTransportados) que extienden de Vehiculo. 3. Clase gestora 'Flota' que gestiona una colección de Vehiculo con relación de agregación/composición (1 a 0..*), métodos de cálculo consolidados (getTotalBono, getTotalCosto, getNumCamiones, getNumAutobuses) y reporte(). Muestra claramente la visibilidad (- privada, + pública), tipos de retorno y estereotipos abstractos.
```

Iteración y Ajustes del Alumno: Se revisó la primera propuesta de la IA asegurando que la clase 'Vehiculo' se representara formalmente con el estereotipo <<abstract>> y su método 'getBonoConductor()' en itálicas/abstracto. Se corrigió la relación entre 'Flota' y 'Vehiculo' para que reflejara agregación/composición de 1 a 0..* en lugar de una simple dependencia.

Resultado Aplicado en el Código: Se validó el modelo estructural completo antes de tocar una sola línea de código, garantizando consistencia conceptual con los requerimientos.

Diagrama de Clases UML Generado y Validado:



Fase 2: Scaffolding (Generación del Esqueleto Base)

Objetivo de la Fase: Construir el scaffolding o esqueleto estructural de Vehiculo, CamionCarga y AutobusPasajeros, implementando atributos privados, constructores y getters/setters, prohibiendo explícitamente métodos de cálculo o toString.

PROMPT UTILIZADO (Interacción con IA):

```
Genera ÚNICAMENTE el esqueleto base (scaffolding) en Java para las clases: 1. 'Vehiculo' (clase abstracta con atributos privados: marca, modelo, capacidad, costoBase, costoTotal). 2. 'CamionCarga' (extiende de Vehiculo, atributos privados: ejes, kilometraje). 3. 'AutobusPasajeros' (extiende de Vehiculo, atributos privados: rutas, pasajerosTransportados). REGLAS ESTRICTAS: - Incluye solo definición de clases, atributos privados, constructores y métodos getters/setters completos. - NO implementes lógicas de cálculo de bonos ni fórmulas. - NO implementes métodos toString(). - Asegura que las subclases invoquen super(...) en sus constructores.
```

Iteración y Ajustes del Alumno: Se verificó que los atributos respetaran estrictamente el encapsulamiento ('private'), que las subclases usaran 'super()' en la primera línea de sus constructores y que no existiera ninguna lógica de negocio prematura.

Resultado Aplicado en el Código: Se obtuvieron esqueletos limpios, desacoplados y listos para recibir la lógica polimórfica en la siguiente fase.

Fase 3: Implementación de Lógica y Polimorfismo

Objetivo de la Fase: Implementar la lógica de negocio y polimorfismo puro mediante el método abstracto getBonoConductor() en Vehiculo y su sobreescritura (@Override) en CamionCarga y AutobusPasajeros, calculando CostoTotal.

PROMPT UTILIZADO (Interacción con IA):

```
Asísteme en la Fase 3 mediante pair-programming guiado para implementar la lógica y polimorfismo: 1. En la clase base 'Vehiculo', declarar el método abstracto: public abstract double getBonoConductor(); 2. En 'CamionCarga', implementar @Override getBonoConductor() con la fórmula: Bono = (CostoBase * 0.10) + (Ejes * 50) + (Kilometraje * 0.5) 3. En 'AutobusPasajeros', implementar @Override getBonoConductor() con la fórmula: Bono = (CostoBase * 0.15) + (Rutas * 120) + (PasajerosTransportados * 0.1) 4. En los constructores de ambas subclases, calcular CostoTotal: setCostoTotal(getCostoBase() + getBonoConductor()); 5. Implementar toString() en Vehiculo y sobreescribirlo en las subclases reutilizando super.toString(). Revisa que la sintaxis de super, modificadores de acceso y tipos de datos sean exactos.
```

Iteración y Ajustes del Alumno: Se depuró la llamada al cálculo de bono dentro del constructor: al invocar getBonoConductor() en el constructor de la subclase, se verificó que los atributos propios (ejes, kilometraje, rutas, pasajeros) ya hubieran sido asignados previamente en 'this' para evitar cálculos con ceros. Asimismo, se formateó CostoBase a dos decimales con String.format(Locale.US, "%.2f", costoBase).

Resultado Aplicado en el Código: Polimorfismo funcional implementado con sobreescritura correcta, reutilización de código mediante super.toString() y cálculo matemático preciso de bonos y costos operativos.

Fase 4: Integración y Ejecución (Gestor y Pruebas)

Objetivo de la Fase: Crear la clase gestora Flota con List, implementar agregación, cálculos acumulados polimórficos sin instanceof en finanzas, conteo de vehículos, reporte() con for-each y probar con la clase App.

PROMPT UTILIZADO (Interacción con IA):

```
Implementa la clase gestora 'Flota' y la clase de pruebas 'App': 1. 'Flota' debe almacenar los vehículos en una colección List vehiculos = new ArrayList<>(). 2. Métodos de cálculo: - getTotalBono(): debe iterar la lista y sumar v.getBonoConductor() POLIMÓRFICAMENTE (¡PROHIBIDO usar instanceof aquí!). - getTotalCosto(): debe iterar y sumar v.getCostoTotal() polimórficamente. - getNumCamiones() y getNumAutobuses(): contabilizar por tipo con instanceof. - reporte(): imprimir cabecera '>> Reporte Detallado de la Flota <<' e iterar con for-each imprimiendo cada vehículo. - toString(): resumen de la flota con totales. 3. Clase 'App' con main() que inserte los datos de prueba del examen: - Flota: 'División Norte', 'Monterrey' - Camión 1: Volvo, FMX, 30000, 2500.00, 4 ejes, 5000.0 km - Camión 2: Kenworth, T680, 25000, 1800.00, 3 ejes, 4200.0 km - Autobús 1: Mercedes, O500, 50, 1200.00, 5 rutas, 8500 pasajeros - Autobús 2: Irizar, i6, 40, 1500.00, 8 rutas, 12000 pasajeros - Autobús 3: Scania, K360, 45, 950.00, 4 rutas, 6200 pasajeros Verifica que la salida en consola coincida con el planteamiento del profesor.
```

Iteración y Ajustes del Alumno: Se optimizó el método reporte() asegurando un salto de línea antes del reporte final y validando que el resumen de Flota mostrara 'Vehículos=5', 'BonoTotal=10637.5', 'CostoTotal=18587.5', 'Camiones=2', 'Autobuses=3'.

Resultado Aplicado en el Código: Integración exitosa de la colección polimórfica en Flota y ejecución del programa App con salida idéntica al requerimiento.

Fase 5: Code Review y Explicación Técnica (Late Binding)

Objetivo de la Fase: Presentar el código a la IA para un Code Review de calidad y documentar en App.java la explicación técnica formal sobre Late Binding (enlace dinámico) en la JVM.

PROMPT UTILIZADO (Interacción con IA):

```
Realiza un Code Review exhaustivo de la solución completa (Vehiculo, CamionCarga, AutobusPasajeros, Flota, App). Analiza: 1. Principios SOLID (especialmente Open/Closed Principle y Liskov Substitution Principle). 2. Manejo de tipos polimórficos en colecciones genéricas. 3. Explicación técnica de bajo nivel: ¿Cómo determina la JVM qué versión de getBonoConductor() ejecutar en el ciclo for-each (Late Binding / Enlace Dinámico)? Explica la instrucción invokevirtual y la tabla virtual de métodos (vtable).
```

Iteración y Ajustes del Alumno: La IA confirmó que el código cumple con OCP y LSP al 100%. A partir de la retroalimentación, el alumno redactó con sus propias palabras un extenso bloque de comentarios técnicos al inicio de App.java detallando la distinción entre early binding (tiempo de compilación) y late binding (tiempo de ejecución con invokevirtual y vtable).

Resultado Aplicado en el Código: Código fuente documentado exhaustivamente y comprensión profunda del mecanismo de despacho dinámico en Java.

Evidencia de Compilación y Ejecución en Consola (Salida Exacta)

Se compiló y ejecutó el proyecto en la terminal mediante javac -J-Xmx128m *.java y java -Xmx128m App obteniendo resultados idénticos a los solicitados en la rúbrica:

```
CONSOLE OUTPUT:
--- Reporte Inicial ---
Flota [Nombre=División Norte, Ciudad=Monterrey, Vehículos=0, BonoTotal=0.0, CostoTotal=0.0, Camiones=0, Autobuses=0]

--- Reporte Final con Datos de Prueba ---
>> Reporte Detallado de la Flota <<
CamionCarga [Vehiculo [Marca=Volvo, Modelo=FMX, Capacidad=30000, CostoBase=2500.00], Ejes=4, Kilometraje=5000.0,
Bono=2950.0, CostoTotal=5450.0]
CamionCarga [Vehiculo [Marca=Kenworth, Modelo=T680, Capacidad=25000, CostoBase=1800.00], Ejes=3, Kilometraje=4200.0,
Bono=2430.0, CostoTotal=4230.0]
AutobusPasajeros [Vehiculo [Marca=Mercedes, Modelo=O500, Capacidad=50, CostoBase=1200.00], Rutas=5,
PasajerosTransportados=8500, Bono=1630.0, CostoTotal=2830.0]
AutobusPasajeros [Vehiculo [Marca=Irizar, Modelo=i6, Capacidad=40, CostoBase=1500.00], Rutas=8,
PasajerosTransportados=12000, Bono=2385.0, CostoTotal=3885.0]
AutobusPasajeros [Vehiculo [Marca=Scania, Modelo=K360, Capacidad=45, CostoBase=950.00], Rutas=4,
PasajerosTransportados=6200, Bono=1242.5, CostoTotal=2192.5]
Flota [Nombre=División Norte, Ciudad=Monterrey, Vehículos=5, BonoTotal=10637.5, CostoTotal=18587.5, Camiones=2,
Autobuses=3]
```

Reflexión del Estudiante sobre el Desarrollo Asistido por IA

La realización del proyecto bajo la metodología gradual de 5 fases permitió evidenciar que la Inteligencia Artificial es sumamente potente cuando se guía como un asistente técnico especializado (pair-programmer) y no como un generador de soluciones monolíticas sin supervisión. Dividir el desarrollo en fases (Modelado → Scaffolding → Lógica/Polimorfismo → Gestor de Colecciones → Code Review) garantizó que cada decisión de diseño fuera auditada, evitando errores frecuentes como la ruptura del encapsulamiento, la duplicación de código o el uso incorrecto de condicionales de tipo (instanceof) que destruyen el polimorfismo. En particular, comprender cómo la JVM efectúa el enlace tardío (late binding) a través de la instrucción de bytecode *invokevirtual* y la tabla de métodos virtuales (vtable) proporciona una sólida base conceptual sobre por qué la Programación Orientada a Objetos permite construir sistemas desacoplados, altamente mantenibles y fácilmente extensibles.